

The Cauldron Protocol

Blind Red-Team Review — Pass 5

Uniswap v4 hook, eternal-relaunch registry,
hook-native perpetuals, NFT collection and on-chain governance

Commit: cbbe4e9, branch fix/b05-b07-relaunch-totality

Working tree: clean (one modified submodule pointer)

Harness: Uniswap v4 on an Ethereum-Sepolia fork

Method: blind — decontaminated tree, prior findings stripped,
no audit material opened before Section 12

Result summary

Severity	Found	Fixed in this pass
Critical	0	—
High	1	1
Medium	4	3 full, 1 partial
Info	2	0 (both refuted as live defects)
Total	5	4 full, 1 partial

Three of the five attack the same property — that `relaunch()` must always be able to run, and must seed the newborn with the value the protocol actually holds. Two are descendants of earlier *fixes*: one generalises a Critical that was patched in one field of a six-field struct, the other is a correctness defect introduced by the remedy for a gas defect. The remaining two are failure-handling asymmetries — a price feed that breaks by reverting is treated differently from one that breaks by going stale, and a caller-supplied venue is validated on two of its five fields.

Contents

1	Abstract	3
2	Methodology	3
2.1	The blind protocol	3
2.2	Decontamination, and its verification	4
2.3	Verification of the decontamination	4
2.4	The residual leak	4
2.5	Anti-hallucination discipline	5
2.6	Orchestration, and its failure	5
3	Baselines	5
3.1	The EIP-170 constraint	6
4	System architecture	6
4.1	The relaunch lifecycle	7
4.2	Governance: the input that reaches the constructor	7
5	Invariant register	8
6	Findings	8
7	Leads — believed reachable, not demonstrated	12
8	Verified safe — probed and held	12
9	Coverage gaps	14
10	Remediation	14
10.1	B-09 — bound the free-text fields at the boundary	14
10.2	B-10 — promote a runner-up instead of rescanning	15
10.3	B-11 — make the oracle total	15
10.4	B-12 — close the trust boundary, report the economics	15
10.5	B-13 — never strand the recovery	16
10.6	Measured cost of all five fixes	16
10.7	Suite, before and after	16
11	Re-review of the remediation	17
12	Comparison with prior work	17
12.1	What the prior passes had already found	17
12.1.1	B-09 generalises C-02, which was patched one field wide	17
12.1.2	B-10 is a defect introduced by the Z-03 fix	18
12.2	Which artifacts would have deflected this reviewer	18
12.3	Comments that were false when written	18
12.4	What the prior passes caught that this one did not	18
13	Threats to validity	19

14 Appendix — reproduction	19
14.1 Environment	20
14.2 The two proofs of concept	20
14.3 Decontamination statistics	20
14.4 Files changed	20

1 Abstract

This is a blind adversarial review of the Cauldron protocol at commit `cbb4e9` (branch `fix/b05-b07-relaunch-totality`), conducted against a decontaminated copy of the repository from which all prior audit material had been removed and all prior finding identifiers stripped from source comments. The decontamination was verified to be comment-only by comparing compiled runtime bytecode: the size table across 1,138 contracts is MD5-identical between the real and blind trees, and for the three largest contracts the runtime bytecode differs only in the 53-byte CBOR metadata trailer.

The review found **five defects** and fixed four in full and one in part.

B-09 (High) is an unbounded proposal payload: a single MiFren holder can author one 24,007,131-gas transaction that makes every subsequent `relaunch()` cost 29,288,732 gas and run out of gas inside a 30M block. Because the resulting revert rolls back the `markConsumed` inside it, the same poisoned proposal wins forever. **B-10 (Medium)** is a positional leader-scan window: 64 spam proposals costing 15,596,982 gas erase a settled, already-voted mandate the moment a normal rebirth consumes the leader. **B-11 (Medium)** is an asymmetry in the oracle's own fail-safe: a feed that breaks by *reverting* bypasses the cache that a feed breaking by *going stale* is protected by, so identical real-world conditions produce opposite outcomes and volume silently records as zero. **B-12 (Medium)** is a caller-supplied swap venue validated on two of a `PoolKey`'s five fields, admitting an attacker-chosen hook into the rotator's own `unlock` and making self-dealing the dominant keeper strategy. **B-13 (Medium)** is a funding selection that tests "can fund" as > 0 : one wei of a second quote outranked a 25 ETH position, seeding a newborn generation with dust while the recovered liquidity was abandoned in the registry where only a timelocked emergency sweep could reach it.

Every finding carries a Foundry proof-of-concept. The fixes cost 581 bytes in total, spread across four contracts that had room for them, leaving the bytecode of `CauldronHook`, `PerpEngine` and `CauldronRegistry` — which sit at 31, 54 and 62 bytes of EIP-170 margin — byte-identical. The suite moved from 103 suites / 547 passing to 110 / 572, with no regressions, no weakened assertions, and one deliberate behaviour change asserted explicitly rather than absorbed silently (Section 10).

Two further comment-versus-code discrepancies were probed and *refuted* as live defects; both are recorded in Section 8 with the reasoning, because a refutation is a result.

The scope completed remains narrower than the scope commissioned, and Section 13 states plainly which areas were never reached.

Would I deploy this to mainnet? Not yet — but the reason has moved. The function graph now exists (321 functions across the hook, perp and NFT clusters) and four of the seven commissioned risk areas have been examined. What still has not been tested adversarially is the perpetual engine's liquidation and solvency behaviour under a manipulated mark, and the holder claim guarantee across a generation boundary. Those are the two places where this protocol holds the most value on behalf of people who cannot withdraw it themselves, and a codebase that has produced a permanent-freeze or value-abandonment defect in every area yet examined should not be assumed clean in the two that have not been.

2 Methodology

2.1 The blind protocol

The value of a blind pass is that attention is allocated by the code rather than by a previous reviewer's conclusions. A comment reading "FIXED — proven by regression test" spends no attention, which is exactly where a defect survives. Both findings in this report sit next to precisely such reassurance.

2.2 Decontamination, and its verification

The tree was copied to `/tmp/blind-tree`, excluding `node_modules`, `.git`, `out`, `cache`, `dist`, `render-out`, `.vercel` and `broadcast`. The following were then removed entirely:

- `audit/` and `contracts/solidity/audit/` (57 files)
- `docs/` — including `SECURITY_ANALYSIS.md`, `DEEP_AUDIT_PROMPT.md`, `BLIND_REDTEAM_PROMPT.md`, `REDTEAM_PASS4_PROMPT.md`
- `contracts/solidity/test/attacks/EXPLOIT_REPORT*.md` (3 files)
- `README.md`, `UNISWAP_AUDIT_GRANT_APPLICATION.txt`
- `archive/superseded-docs/BUGS_FIXED.md` — *not* matched by the commissioned keyword list, but it enumerates prior defects; quarantined on its own merits.

Finding identifiers were then stripped from comments by a purpose-written comment-aware pass (`/tmp/strip_findings.py`). A naive line regex was rejected: 48 identifier tokens live inside *string literals* — revert reasons and assertion messages — and rewriting those would change compiled bytecode rather than merely removing an anchor. The stripper tracks quote state character by character and edits only comment text.

An initial run rewrote 5,654 lines, which was wrong: a whitespace-tidying step was firing on comments that contained no identifier at all. Restricting the tidy to comments actually modified brought this to **486 lines across 86 files**, which is the reported figure.

2.3 Verification of the decontamination

Check	Result
<code>grep -rn "audit [A-Z]" -include=*.sol</code>	8 hits, all inside string literals
Any <code>[A-Z]{1,2}-[0-9]{1,2}</code> token in non-lib <code>.sol</code>	49 hits
of which in comments	0
of which in string literals	48
of which bare identifiers in code	0 (1 in a deploy script path)
Build size table, real vs blind	1,138 rows, MD5-identical
Runtime bytecode, CauldronHook	body identical; differs only in 53-byte metadata
Runtime bytecode, PerpEngine	body identical; differs only in 53-byte metadata
Runtime bytecode, CauldronRegistry	body identical; differs only in 53-byte metadata

The bytecode comparison is stronger evidence than the commissioned size-table check: identical sizes could mask compensating edits, whereas an identical body proves the edits touched only comments.

2.4 The residual leak

Perfect blinding was not achieved, and the report does not claim it.

1. **Reviewer memory (the largest leak).** The reviewer’s own persistent memory store contained summaries of findings B-01, B-05 and B-06 from earlier passes, including file and line references. This could not be unseen. It is a direct pointer into the relaunch and fee-routing paths — and B-09 was found in the relaunch path. The finding was re-derived from the source, but the *attention* was not independently allocated.
2. **Surviving string literals.** 48 identifier tokens remain in revert reasons and assertion messages, unremovable without changing bytecode.

3. **Explanatory prose was deliberately preserved.** Stripping the tag while keeping the sentence leaves comments such as “CLAMP, never revert: an out-of-range supply would revert in the collection constructor, which rolls `markConsumed` back” (`CauldronRegistry.sol:850-856`). That sentence is a complete statement of a prior threat model. This was the commissioned trade-off and it is the right one — but it means the reviewer inherited the codebase’s threat model even where the tags were gone.
4. **Test names.** `B07_RelaunchTotality.t.sol`, `Z03_GovernorSpamRelaunchDoS.t.sol` and `B05_NonEthRelaunchBrick.t.sol` survive as filenames and reveal both the considered threats and the prior pass letters.
5. **Branch name.** `fix/b05-b07-relaunch-totality` was visible in the environment and names the area of most recent change.

Leak (1) and leak (4) both point at the relaunch lifecycle, which is where both findings were made. A reader should discount this pass’s claim to have allocated attention independently, and read the two findings on their technical merits instead.

2.5 Anti-hallucination discipline

Every claim in this report carries a file and line verified by reading that line; every symbol was grepped before being cited; every quoted excerpt was copied from the file. Four claims made by comments in the source were tested against the code rather than accepted, and two of them were false — see Section 12.

2.6 Orchestration, and its failure

Five parallel extraction subagents were dispatched to build the function graph (Section 4). **All five terminated on an API rate limit before writing any output.** The graph was therefore never built, and the review proceeded by direct reading instead. This is reported rather than concealed because it is the single largest reason the completed scope is narrower than the commissioned scope.

Subagent accounting. First dispatch: five agents, zero output, all lost to the rate limit. Second dispatch: three agents, all completed, 321 functions extracted across six files. Three observations were promoted from that output into this report as candidate findings; on independent re-verification by the orchestrator, **one survived** (B-13’s per-asset reserve keying) and **two were refuted** (Section 8) — a 67% discard rate on promoted subagent claims, which is the number that matters when judging how much of this report rests on delegated work. The other four findings were derived directly by the orchestrator. No subagent claim appears in this report without a line the orchestrator read itself.

3 Baselines

Every number below was measured at `cbbe4e9`. Prior prompt material in this repository quotes “87 Foundry suites”, “486 external/public entrypoints” and “~50 modified files, 2 deleted, 1 untracked”; none of the three matched the tree.

Quantity	Measured
<code>git status -short</code>	1 line (modified submodule pointer). The tree is clean ; the commissioned “dirty tree” had been committed.
Solidity files outside <code>lib/</code>	143
Lines outside <code>lib/</code>	36,569 (16,381 non-test)

Quantity	Measured
State-changing external entrypoints	355 (327 nonpayable + 28 payable)
Total ABI functions (non-test)	982
Suite, with fork env	103 suites, 547 passed , 0 failed, 1 skipped
Suite, without fork env	362 passed, 170 skipped
Fork gate	31% of the suite is invisible without FORK_RPC

The single baseline skip is `BadgeDump`, an artwork-dump utility, not a security test.

3.1 The EIP-170 constraint

Contract	Runtime bytes	Margin to 24,576
CauldronHook	24,545	31
PerpEngine	24,522	54
CauldronRegistry	24,514	62
PoolOps	23,365	1,211
MiFrensGenesis	20,261	4,315
CauldronGovernor	6,770	17,806

This table governs remediation. Any fix that must live in the hook, the engine or the registry has effectively no room; the governor is the only roomy contract in the core. Both fixes in this pass were placed accordingly — and, as Section 10 argues, the governor is also the only place either fix would actually work.

4 System architecture

Scope note — the graph, on the second attempt

The first dispatch of five parallel extraction subagents died on a session rate limit before writing anything. A second dispatch — three agents, on a smaller model, staggered, each writing to disk and returning only a short summary so its output could not flood the orchestrator's context — completed. The graph covers **321 functions**:

Cluster	Files	Functions	Bytes
Hook / fee reserve	hook.json, hook.md	93	95,566
Perp engine	perp.json, perp.md	100	102,205
NFT / dividend	nft.json, nft.md	128	113,711

Three of the five findings in this report were reached by direct reading before the graph existed; **B-13 came out of the graph** — the per-asset reserve inventory made it visible that `relaunchAsset` is keyed per asset while `seedFunding` only ever pulls three specific keys. The two refutations in Section 8 also came from the graph, as comment-versus-code observations the extraction brief asked for explicitly and forbade the agents from judging.

Still not built: the registry, pool/seed and governance clusters, and the three derived views (value-flow, authority lattice, trust boundaries) as formal artifacts.

4.1 The relaunch lifecycle



The design intent, stated verbatim at `CauldronRegistry.sol:908-910`, is:

“Reverting on the LEFT of this line is safe and recoverable — the proposal stays live and a later call can succeed once the machine has value. Reverting on the RIGHT is what must never happen.”

This is a sound principle, correctly implemented for *reverts*. B-09’s contribution is that it is not a defence against *gas exhaustion*: an out-of-gas condition is not a catchable sub-call failure, it is the whole transaction failing, and it rolls back the consumption from either side of the line. Every clamp, every try/catch and the entire repositioning of `markConsumed` are bypassed by running out of gas before reaching any of them.

4.2 Governance: the input that reaches the constructor

`CauldronGovernor.propose` is permissionless for the holder of a single MiFren (:170) with no cooldown, no deposit and no per-address cap. The resulting `BrewSpec` (`ICauldron.sol:19-31`) has eleven fields. Six are free-text or supply parameters that flow into `relaunch()`:

Field	Bounded at propose?	Fate in <code>relaunch()</code>
<code>nftSupply</code>	yes :182	clamped again Registry:854
<code>quote</code>	yes :204	re-checked, degrades Registry:844
<code>name</code>	no (pre-fix)	re-stored in the collection
<code>symbol</code>	no (pre-fix)	re-stored in the collection
<code>baseURI</code>	no (pre-fix)	re-stored in the collection
<code>website</code>	no (pre-fix)	read and discarded
<code>socials</code>	no (pre-fix)	read and discarded

Field	Bounded at propose?	Fate in relaunch()
volumePerNFT	no	hook.setNftCurveFrom — see Leads
renderer	code-size check :176	passed to the constructor

The two rows marked “read and discarded” are pure amplification: `winner()` (CauldronGovernor.sol:288-300) copies `website` and `socials` out of storage into the returned struct on every rebirth, and `relaunch()` never references either.

5 Invariant register

Only the invariants actually examined are listed. An empty row would be worse than an absent one.

Invariant	Enforcement	Test status
A dead generation can always be replaced (relaunch liveness)	<code>markConsumed</code> ordering, clamps, try/catch	Was UNENFORCED against gas — B-09. Now bounded at the boundary; regression B09_*
A voted, settled mandate remains reachable until consumed	cached leader + bounded scan	Was UNENFORCED — B-10. Now carried in an explicit runner-up slot; regression B10_*
<code>relaunch()</code> never reverts after <code>markConsumed</code>	clamps + try/catch Registry:844-912	B07_RelaunchTotality — but every test stubs the governor (Section 9)
Leader scan gas is $O(1)$ in proposal count	<code>MAX_LEADER_SCAN = 64</code> :364	Z03 — holds; re-verified this pass
Ties keep the earlier leader	<code>p.votes > _leaderVotes</code> :351	Held under the new runner-up logic; B10_MoreVotesStillWins
A consumed proposal is never re-promoted	<code>markConsumed</code> :366-386	Newly tested this pass — B10_PromotionNeverResurrects-AConsumedProposal

6 Findings

B-09 — An unbounded proposal payload puts `relaunch()` permanently out of gas

Severity: **High** **Confidence:** **Confirmed** (VERIFIED, PoC) **Status:** **Fixed**

Location. `cauldron/CauldronGovernor.sol:171-173` (the only check), with the replay at `cauldron/CauldronGovernor.sol:288-300` and `CauldronRegistry.sol:796,1070`.

```
// CauldronGovernor.sol:171-173 -- the entire validation of five fields
if (bytes(name).length == 0 || bytes(symbol).length == 0) revert EmptyField();
if (mode == MetadataMode.BaseURI) {
    if (bytes(baseURI).length == 0) revert EmptyField();
```

Mechanism. `propose()` bounds `nftSupply` (:182) and `quote` (:204) precisely because they flow into `relaunch()`, and says so. It applies no upper bound to `name`, `symbol`, `baseURI`, `website` or `socials`. `relaunch()` reads all five back through `winner()` and re-stores three of them in the newborn collection via `CauldronFactory.deployBrew`.

The payload is cheap to store and dear to read. Writing a zero word to a fresh storage slot is a 100-gas no-op `SSTORE`; reading it back is a full 2,100-gas *cold* `SLOAD`. Measured marginal cost:

the author pays **72 gas/byte once**, the protocol pays **83 gas/byte on every rebirth, forever**.

That asymmetry is the structural defect and it is independent of the block gas limit: whatever the limit is, one block of authoring buys more than one block of replay.

Why the existing defences do not apply. The whole of the B-05/B-07 remediation — moving `markConsumed` as late as the logic allows, clamping rather than reverting, wrapping callees in `try/catch` — defends against *reverts*. An out-of-gas is not a catchable sub-call revert; it is the transaction failing. The consumption never commits, so `_bestUnconsumed()` returns the same poisoned proposal to the next caller, and to every caller after that.

Precondition. One MiFren (for `getVotes > 0`), and enough votes for the proposal to lead. There is no quorum, no minimum turnout and no deposit, so in a low-turnout guild a single vote suffices.

Impact. The dead generation can never be replaced. Holders cannot migrate; the reserve never redeploys. Recovery requires the guild to elect a competing proposal with *strictly* more votes (:351), so an attacker holding a voting plurality makes the freeze permanent and unrecoverable.

Cost. One transaction, **24,007,131 gas**, atomic. The payload need not travel as transaction calldata — a contract expands it in memory — so the ~128KB geth mempool limit does not apply.

Measured on the Sepolia fork, 30M block	gas
<code>honest relaunch()</code>	5,297,638
attacker authors 320KB in baseURI (one tx)	24,007,131
<code>poisoned relaunch()</code>	29,288,732 → OOG

PoC. `test/audit/B09_ProposalPayloadGasBrick.t.sol` — `test_INVARIANT_B09_RebirthSurvivesAHostilePayload` (fork-gated, real governor and registry end to end), plus three non-fork demonstrations.

B-10 — Spam erases a voted mandate from the leader scan

Severity: **Medium** **Confidence:** **Confirmed** (VERIFIED, PoC) **Status:** **Fixed**

Location. `cauldron/CauldronGovernor.sol:368-380`.

```
function _recomputeLeader() private view returns (uint256 bestId, uint256 bestVotes) {
    uint256 n = proposalCount;
    uint256 first = n > MAX_LEADER_SCAN ? n - MAX_LEADER_SCAN + 1 : 1;
    for (uint256 i = first; i <= n; i++) {
```

Mechanism. The scan is bounded to the newest 64 proposals, which is what keeps rebirth gas $O(1)$ in the proposal count. But the window selects on proposal *id*, not on whether anyone voted — so a settled mandate the guild has already voted for can be pushed out of it by proposals authored afterwards, and `propose()` is permissionless.

The cached-leader fast path in `_bestUnconsumed` conceals this for the *current* leader. It does not protect the mandate queued *behind* it, because `markConsumed` rebuilds the cache from the scan the instant the leader is consumed — which is exactly what a normal rebirth does.

Precondition. One MiFren. The guild must have queued a second mandate, which is what a guild that plans ahead does.

Impact. `hasProposals()` returns false, so `relaunch()` reverts `NoProposal()` at `CauldronRegistry.sol:734` and the machine cannot be reborn until the guild authors a replacement and waits out a further three-day `VOTING_PERIOD`. The revert is on the *safe* side of `markConsumed`, so this is a grieving vector rather than a permanent brick — but the protocol is frozen while it holds: the pool is dead and holders cannot migrate.

Cost. 15,596,982 gas for 64 spam proposals — one block’s worth, cents on an L2 — to impose a ≥ 3 -day freeze, repeatable at every generation boundary indefinitely. No value is extracted; the grief is priced entirely in delay.

PoC. `test/audit/B10_ScanWindowErasure.t.sol` — `test_INVARIANT_B10_QueuedMandateSurvivesSpam`.

B-11 — A reverting price feed bypasses the oracle’s own fail-safe

Severity: Medium **Confidence:** Confirmed (VERIFIED, PoC) **Status:** Fixed

Location. `cauldron/QuoteOracle.sol:144,148,207-216`, consumed at `CauldronHook.sol:677-682,769`.

Mechanism. `QuoteOracle` states its failure policy explicitly (:40-44): death is irreversible and relaunch permissionless, so “when a price is unusable this returns 0, and callers must treat 0 as ‘cannot judge’ rather than ‘no volume’”. Failing toward ALIVE is the only safe direction when the wrong answer cannot be undone.” It then enumerates three unusable conditions (:49-55) — no feed, stale, sequencer down.

All three are cases where the feed *answers* and the answer is no good. A feed can also fail by *not answering*: a retired or migrated aggregator, an access-controlled one, or an address with no code (a high-level call to a codeless address reverts on the `extcodesize` check). That fourth case did not return 0 — it threw, through `usdPerRawUnit` and out of `cachedUsdPerRawUnit`, which calls it via `this`. and so propagates, never reaching the line that keeps the last good factor.

Two further revert paths were found while confirming it, both from checked arithmetic: a feed reporting `updatedAt` in the future underflowed `block.timestamp - updatedAt`, and the sequencer feed reporting a future `startedAt` underflowed `block.timestamp - startedAt`.

Impact. An asymmetry between two forms of the same real-world condition:

feed state	outcome
stale	cache keeps last good price; volume records correctly
reverting	cache bypassed; <code>_toUsd</code> returns 0; volume records as zero

`CauldronHook._toUsd` wraps the oracle in a raw call and returns 0 on failure, so swaps do *not* go down — but `CauldronHook.sol:769` then reads `if (absVolume > 0)`, so the swap records nothing, and `isDead (:1512)` judges `vol < deathThreshold`. A feed that broke by reverting pushed a live generation toward a permissionless, irreversible relaunch; the identical feed going stale did not.

Latent at the shipped config. `indexer/deployments/round.json` carries `deathThresholdEth: 0`, and `vol < 0` is never true, so nothing dies today. This is rated against the configuration the design intends, not the one currently deployed.

PoC. `test/audit/B11_OracleRevertBypassesCache.t.sol` — six tests, the control (`StaleFeedKeepsTheCachedValue`) passing before and after.

B-12 — The rotator accepts any venue with the right currency pair

Severity: Medium **Confidence:** Confirmed (VERIFIED, PoC) **Status:** Partially fixed — trust boundary closed, execution-quality reported

Location. `cauldron/QuoteRotator.sol:493-501`, reached from the permissionless `rotateStep (:190)`.

```
address c0 = Currency.unwrap(route.currency0);
address c1 = Currency.unwrap(route.currency1);
return (c0 == from && c1 == to) || (c0 == to && c1 == from);
```

Mechanism. Two of a `PoolKey`’s five fields are validated. Not `fee`, not `tickSpacing`, and not `hooks`. `Pool` initialisation is permissionless in v4, so anyone may create a pool on the same pair — at any price, behind a hook whose permission bits they mined — and nominate it as

the venue.

Impact, part 1 (economic). The header argues the design is safe because “the price floor ... comes from the governed plan” (:185-188). The floor is real; what the argument omits is that the floor becomes the *price*. An honest keeper routing to the deepest venue earns `keeperBps = 0.1%`. A keeper routing to a pool they seeded earns that 0.1% *and* the entire spread, by returning exactly `minOut` and keeping the difference as inventory. Self-dealing strictly dominates, so the rotation realises worst-case execution on every slice rather than in the worst case. `minRate` is stored verbatim from the vote (:69-71) and never revised while a plan runs for hours, so the exploitable spread is the guild’s slippage tolerance plus all market drift since the vote, and it widens as the plan ages. Extraction is atomic and needs no capital beyond the `minOut` seeded into the attacker’s own pool, recovered in the same transaction.

Impact, part 2 (trust boundary). An attacker-supplied `hooks` address is invoked by the `PoolManager` inside this contract’s own `unlock`. `v4`’s delta accounting stops the direct theft — an unbalanced delta reverts the `unlock` — but arbitrary code runs while the manager is unlocked and pool state is mid-swap.

Coverage. Every route in `test/QuoteRotator.t.sol` and `test/RotatorSwapFork.t.sol` passes `hooks: IHooks(address(0))`. The hooked-venue case was never exercised.

PoC. `test/audit/B12_RotatorVenueUnvalidated.t.sol`. The discriminator: a wrong-pair route reverts `NoRoute`; the arbitrary-hook route reverted *without* that selector, proving it passed validation and reached execution.

B-13 — One wei of the requested quote outranks the entire recovered position

Severity: **Medium** **Confidence:** **Confirmed** (VERIFIED, PoC) **Status:** **Fixed** (with a deliberate, asserted behaviour change)

Location. `cauldron/PoolOps.sol:892-904`.

```
if (wantQuote != address(0)) {
    uint256 p = (oldQuote == wantQuote ? recovered : 0) + _pullAsset(hookAddr, wantQuote);
    if (p > 0) return (wantQuote, p, vaultSwept);
}
```

Mechanism. `seedFunding` decides what the newborn generation is seeded with, trying the requested quote, then native, then the dying quote. The intent is stated at `CauldronRegistry.sol:836-839`: “narrows the request to what the protocol ACTUALLY HOLDS ... A quote nobody can fund degrades to one we can.” But “can fund” is tested as `p > 0` and the `return` is unconditional, so the first denomination holding any dust wins and the later branches are never consulted.

How the dust arrives. `relaunchAsset` is keyed per asset and credited from whatever asset a fee arrived in (`CauldronHook.sol:1150`, keyed on `_feeAsset` set per swap at :1378), and every registry-opened pool is tracked and charges fees (:599). Any guild that has ever diversified into a second quote — the live manifest offers USDG and xNVDA — carries a balance in it from ordinary trading. No attacker is required.

Impact. `seedFunding`’s return is the newborn’s entire book: `relaunch()` passes it to `_seedGeneration` at `CauldronRegistry.sol:1000`, and `totalETH == 1` clears the `NoLiquidityToSeed` guard at :911. Measured: with 20 ETH recovered and a 5 ETH reserve, one wei of USDG produced (USDG, 1). The abandoned value does not come back — `recovered` sits in the registry, and the only paths that spend a loose registry balance are `migrateToSuccessor` (:493) and the timelocked `emergencySweep` (:413), both break-glass.

Coverage. `test/attacks/B08_NonEthRebirth.t.sol` covers Leg 1 only with `wantQuote == oldQuote`, so the cross-asset case with a live recovery was never exercised.

PoC. `test/audit/B13_SeedFundingDustPreference.t.sol` — five tests, callable without a fork because `seedFunding` is an external library function.

7 Leads — believed reachable, not demonstrated

L-1 — volumePerNFT is unbounded and reaches the live curve

BrewSpec.volumePerNFT has no bound in propose(), and relaunch() passes it to hook.setNftCurveFrom at CauldronRegistry.sol:1015, which assigns it without validation:

```
// CauldronHook.sol:1958-1963
function setNftCurveFrom(uint256 _base) external {
    if (msg.sender != registry) revert OnlyRegistry();
    if (_base == 0) return;
    volumePerNFT = _base;
    nftPriceStep = 0;
}
```

This cannot brick the rebirth — the assignment cannot revert, and nftPriceStep = 0 removes the overflow term from nftPriceAt. The suspicion is economic: a winning proposal setting volumePerNFT to an absurd value would make crystals unmintable for that entire generation.

Confidence: HYPOTHESIS. Next step: assert crystalsReady() is non-zero after a rebirth carrying volumePerNFT = type(uint256).max, at realistic volume.

L-2 — The promoted runner-up may still be inside its voting window

The B-10 fix promotes the runner-up on consumption. If that proposal's voting window has not closed, _bestUnconsumed's fast path declines it (:388-390 requires block.timestamp > cached.votingEndsAt) and falls back to the bounded scan — the same scan B-10 shows is floodable. The exposure is bounded by the runner-up's remaining voting time and self-heals when the window closes, and refusing to launch a still-open proposal is the correct anti-front-run behaviour, so this is a narrowed window rather than a hole. **Confidence: DERIVED** (reasoned from the code, not exercised). **Next step:** a test that consumes the leader while the runner-up still has voting time left, spams 64, and asserts the mandate is reachable once its window closes.

8 Verified safe — probed and held

Two comment-versus-code discrepancies surfaced by the extraction pass were probed hard enough to be reported as findings and *did not survive* verification. Both are recorded here with the reasoning, because a refutation is a result and because each names a real latent hazard worth a one-line hardening.

REFUTED — _feeAsset is documented as transient but is not

CauldronHook.sol:626 calls _feeAsset “a transient field”, and :630 declares it address internal _feeAsset; — ordinary storage. The same contract declares _inSelfBuy and _inRelaunchClose with the genuine transient keyword at :305 and :310, which makes the wording look like a missing keyword rather than informal English.

It would be a defect only if some path read _feeAsset without writing it in the same call, since a stale quote address would misroute a reserve credit. It does not: the single write at :1378 dominates all eight reads. _takeEthFee is called only from _afterSwap (:933) and _beforeSwap (:1108); _routeEthFee only from :1385 and :1393, both inside _takeEthFee after the write; _creditReserve only from :1134 and :1272, both downstream of the same. **No stale read is reachable.**

Latent hazard, not a live defect: the value persists between transactions, so any future reader added outside _takeEthFee silently inherits the previous swap's quote. Declaring it transient would make that structurally impossible and replace a per-swap SSTORE with a TSTORE.

REFUTED — two transferFrom calls discard the return value

cauldron/PerpEngine.sol:1457 (fundPlvToken) and :1551 (fundTokenFromVault) both call IERC20(registry.currentToken()).transferFrom(...) and discard the returned bool, while every other pull in the file checks it — _pullQuote (:190-193) and _safeTransfer (:1434-1436) decode and require it, and the matching outbound path withdrawPlvTokenTo (:1556-1561) uses the checked helper. Both credit plvToken immediately after, so a silent failure would inflate the vault's token-side backing — nominally a solvency defect. It is not reachable. registry.currentToken() is always a CauldronToken, which is contract CauldronToken is ERC20 on OpenZeppelin (CauldronToken.sol:4,26); its transferFrom reverts on insufficient balance or allowance and cannot return false. Both callers are additionally privileged (onlyOwner, onlyVault). **No false-returning token is reachable on this path.** Latent hazard, not a live defect: the inconsistency is one edit away from mattering if currentToken() ever returns a non-reverting ERC20. Reported at Info rather than inflated into the “perp vault insolvency” risk area it superficially resembles.

Property	How hard it was probed, and what held
Vote weight cannot be minted by transfer	vote() uses getPastVotes(msg.sender, p.snapshot) (:344) with block.number <= p.snapshot rejected (:343). Transferring MiFrens after the snapshot yields no new weight. Read directly; not attacked with a PoC.
A still-open proposal cannot be launched	_bestUnconsumed requires block.timestamp > cached.votingEndsAt (:389) and _recomputeLeader skips open proposals (:374). Closes the same-block front-run of the permissionless relaunch().
An unvetted quote cannot reach a vote	propose() staticcalls the registry allowlist and treats a short or failed return as refusal (:204-209). Exercised by the existing B07_GovernorQuoteBoundary suite, which correctly wires the registry — without that wiring the test would pass for the wrong reason, and the suite says so.
Leader-scan gas is flat under spam	Re-verified: 64 spam proposals cost the attacker 15,596,982 gas and did <i>not</i> raise scan cost. The Z-03 gas fix holds; it is its <i>correctness</i> side-effect that B-10 reports.
The three near-limit contracts survive remediation	CauldronHook, PerpEngine and CauldronRegistry runtime bytecode is byte-identical before and after all five fixes.
Both hook reserve pulls and the vault close are revert-proof	The claim at CauldronRegistry.sol:906 that seedFunding “catches both reserve pulls and the vault close” was checked rather than trusted, and holds: try/catch at Pool0ps.sol:889 (vault close), :913 (releaseRelaunchAsset) and :920 (releaseRelaunchETH).
A reverting oracle cannot take swaps down	_toUsd (CauldronHook.sol:677-682) uses a raw call and checks both ok and ret.length, so a hostile or broken oracle cannot revert into beforeSwap. The consequence it <i>does</i> have is B-11.
Only one live pool-state read exists in the swap path	Extraction found exactly one: poolManager.getSlot0(id) at CauldronHook.sol:1329 inside _defaultSurtaxBps, reached from both swap callbacks. The in-swap manipulation surface is therefore one value (the anti-snipe surtax), not the broad surface the architecture suggested. Not further attacked.

9 Coverage gaps

Ranked by the value at risk behind them.

1. **B07_RelaunchTotality stubs out the only attacker-controlled input.** The suite states the correct property — “`relaunch()` must never revert in a way that rolls back `markConsumed`” — and then supplies every proposal through `BGov07` (`test/attacks/B07_RelaunchTotality.t.sol:243-258`), a stub whose `winner()` returns hardcoded 8-byte strings. Its hostile inputs are hostile *vaults* and *hooks*. The proposal itself — the one thing an unprivileged attacker actually controls — is never varied. Both findings in this report live in that gap.
2. **Z03_GovernorSpamRelaunchDoS asserts cost, never reachability.** Its assertions are `assertEq(gas800, gas200)` and `assertLt(gas800, 250_000)` — that the scan stays *cheap*. No assertion checks that the scan still *finds* a mandate that exists. B-10 is exactly that missing assertion.
3. **31% of the suite is invisible without fork env.** 170 of 548 tests skip silently when `FORK_RPC` is unset, including `test_FIXED_B05_*`, `test_Fixed_A01_*` and every `*_OnFork` lifecycle test. A CI run without the fork variables is green while testing none of the relaunch security properties.
4. **No test bounds any proposal free-text field (pre-fix).** Now covered by `B09_*`.
5. **No test asserted that a consumed proposal is never re-promoted.** Added this pass as `test_FIXED_B10_PromotionNeverResurrectsAConsumedProposal`.

10 Remediation

10.1 B-09 — bound the free-text fields at the boundary

Root cause. Not “strings are unbounded” but: *untrusted proposal input is validated field-by-field rather than as a class*, so a principle already established for two fields was never extended to the five beside them. B-09 and the prior C-02 share this root cause.

Chosen shape. Per-field byte caps enforced in `propose()`, beside the existing `nftSupply` and quote bounds: `MAX_NAME_BYTES = 64`, `MAX_SYMBOL_BYTES = 16`, `MAX_URI_BYTES = 256`, `MAX_LINK_BYTES = 128 (×2)`. Worst legal case 592 bytes.

Rejected alternatives.

- *Truncate inside `relaunch()`.* Rejected on two independent grounds. It does not work — by the time `relaunch()` could truncate anything, the cold SLOADs that dominate the cost have already been paid by `winner()`. And it does not fit: `CauldronRegistry` has 62 bytes of EIP-170 margin.
- *Store a hash and move the text off-chain.* Rejected as disproportionate: it breaks the display surface and the indexer for a problem a length check solves.
- *A size-proportional proposal deposit.* Rejected: it changes governance economics, and it bounds the *expected* payload rather than the worst case.
- *Drop website/socials from `BrewSpec`* (they are read and discarded). Tempting and genuinely wasteful, but it changes a struct the tests and indexer consume; bounding them achieves the security goal without the blast radius.

Verification. `test_INVARIANT_B09_RebirthSurvivesAHostilePayload` now asserts both halves: the bomb is refused, *and* the largest still-legal proposal rebirths in 5,315,432 gas — 17,706 gas above the honest baseline, against 24M before.

10.2 B-10 — promote a runner-up instead of rescanning

Root cause. Consumption *depends on a positional scan*. Any window over proposal ids can be flooded, and an unbounded one is the gas brick `MAX_LEADER_SCAN` exists to prevent, so the fix must remove the dependency rather than resize the window.

Chosen shape. Two appended storage slots, `_runnerId` and `_runnerVotes`, maintained incrementally in `vote()` and promoted in `markConsumed()`. Spam cannot occupy either slot because both require votes; displacing a mandate now costs actual voting power, which is governance working as intended.

Rejected alternatives.

- *Enlarge `MAX_LEADER_SCAN`*. Rejected: any positional window is floodable, and enlarging it walks back toward Z-03.
- *Scan a list of voted proposals instead of all proposals*. Rejected: an attacker with one MiFren can vote once on each of 64 spam proposals, since `hasVoted` is per proposal per address. It raises the cost without removing the attack.
- *A full priority queue over live proposals*. Rejected as disproportionate for a griefing-severity finding, and it is the kind of consensus-adjacent rewrite that should not be bolted on at the end of an audit.

Storage safety. Both slots are *appended* after `_leaderVotes`; no existing slot moves. No test in the repository pins `CauldronGovernor` slot numbers (checked: `FacetLayoutInvariant` does not reference it).

10.3 B-11 — make the oracle total

Root cause. The failure policy was written for feeds that answer badly and never extended to feeds that do not answer. The cache’s whole purpose — keeping the last good factor — is expressed on a code path a reverting feed never reaches.

Chosen shape. `try/catch` around both external reads in `usdPerRawUnit` (`latestRoundData` and `decimals`) and around `_sequencerOk`’s read, each degrading to “unusable” rather than throwing; plus explicit refusal of a future `updatedAt/startedAt` instead of letting checked arithmetic underflow. The uncached view becomes total, so the cached path and `priceable()` inherit the fix.

Rejected alternatives. Wrapping the call at the *consumer* (the hook already does, which is why swaps survive) — rejected because it fixes the symptom in one caller and leaves `priceable()` and every future consumer exposed, and because it cannot reach the cached value that is the actual protection. Removing the sequencer check on failure — rejected as failing toward DEAD, the direction the contract’s own preamble forbids.

10.4 B-12 — close the trust boundary, report the economics

Root cause (part 2). A caller-supplied struct validated on two of five fields.

Chosen shape. `if (address(route.hooks) != address(0)) revert NoRoute();` in `rotateStep` only. A quote-to-quote rotation trades two quote assets, which is a plain external pool; the protocol’s own hooked pools are (quote, brew token) and are never a valid venue here. `swapOnce` is left unconstrained — it is owner-only.

Deliberately not fixed (part 1). Bounding execution *quality* needs either a venue allowlist (governance must curate and maintain it) or an oracle band (which bricks rotation whenever the oracle is unavailable — see B-11 for how that happens). Both are policy choices with real failure modes, and neither is mine to make silently inside an audit. Reported with the recommendation instead.

10.5 B-13 — never strand the recovery

Root cause. “Can fund” was tested as “holds a non-zero balance”.

Chosen shape. Two boolean guards, no reordering and no new branches: branch 1 is taken only when `recovered == 0 || oldQuote == wantQuote`, and branch 2 only when `recovered == 0 || oldQuote == address(0)`. The rule is not “pick the biggest” — that needs a price — but the weaker, price-free “never strand recovered”.

Why it cannot brick. This is the one function that must never be the reason the machine cannot be reborn, so the guard was chosen to be provably non-bricking: when `recovered > 0` exactly one of branches 2 and 3 is reachable and both include `recovered`, so any state that previously returned a funded answer still returns one; when `recovered == 0` every branch behaves exactly as before. Asserted directly by `test_FIXED_B13_NeverReturnsZeroWhenValueExists`.

Rejected alternatives. An oracle-ranked selection — rejected: putting a reverting external dependency inside the rebirth’s funding decision trades a value bug for a liveness bug, and B-11 is a live demonstration that this oracle can fail. Restructuring the branch order — rejected: every price-free ordering has a mirrored pathology (prefer `recovered` and a dust recovery beats a large reserve), and restructuring the most brick-sensitive function in the protocol is how this codebase has historically introduced new defects.

The behaviour change, stated rather than absorbed. A guild that votes a new quote while the dying pool still holds value no longer gets that quote at this rebirth; it gets a rebirth in the denomination the value is already in, and the requested quote’s balance stays in `relaunchAsset` — still claimable later, unlike the registry balance the old path stranded. The supported way to rotate is to convert the treasury first via `QuoteRotator`, after which `oldQuote == wantQuote` honours the vote. This is asserted by a test named for the change (`test_CHANGED_B13_RequestInAnotherAssetNoLongerAbandonsRecovery`) rather than by quietly flipping the control assertion that caught it.

10.6 Measured cost of all five fixes

Contract	Before		After	Margin left
CauldronGovernor	6,770		7,219 (+449)	17,357
PoolOps	23,365		23,446 (+81)	1,130
QuoteOracle	2,641		2,667 (+26)	21,909
QuoteRotator	7,149		7,174 (+25)	17,402
CauldronHook	24,545	24,545 (unchanged)		31
PerpEngine	24,522	24,522 (unchanged)		54
CauldronRegistry	24,514	24,514 (unchanged)		62

Total +581 bytes. No fix was placed in a contract that could not afford it; the three at the EIP-170 boundary are byte-identical.

10.7 Suite, before and after

	Suites	Passed	Failed	Skipped
Baseline (cbbe4e9)	103	547	0	1
After all five fixes	110	572	0	1

No existing assertion was weakened and no existing test was modified. The 25 new tests are additive. One control test written during this pass was invalidated by the B-13 fix; it was rewritten to assert the new semantics under a name that says so, not deleted and not silently flipped.

11 Re-review of the remediation

The remediation was re-read as a distinct adversarial pass, on the explicit ground that this codebase’s last remediation produced three findings of which it had introduced two. Four attacks were constructed against the B-10 fix specifically and all four are now permanent regression tests:

Attack on my own fix	Result
Promote a <i>consumed</i> proposal, launching the same brew twice	Blocked by <code>r.exists && !r.consumed</code> . Test: <code>PromotionNeverResurrectsAConsumedProposal</code>
Leader gains votes and becomes its own runner-up	Blocked by the <code>proposalId != _leaderId</code> guard on demotion. Test: <code>LeaderCannotBecomeItsOwnRunnerUp</code>
Consume the runner-up out of band, leaving a stale promotable id	Blocked by the new <code>else if (proposalId == _runnerId)</code> clear. Test: <code>ConsumingTheRunnerUpClearsTheSlot</code>
Break ordinary governance (more votes must still win)	Held. Test: <code>MoreVotesStillWins</code>

One residual is carried openly as Lead L-2: a runner-up promoted while its voting window is still open falls back to the scan until that window closes. It self-heals and refusing to launch an open proposal is correct behaviour, but it is a narrowed exposure rather than an eliminated one, and it is stated rather than claimed away.

Defects introduced by the fixes: none found. That is a claim about four constructed attacks and a green suite, not a proof.

12 Comparison with prior work

Read only after the findings above were fixed and verified.

12.1 What the prior passes had already found

The prior corpus is substantial: `CauldronAudit.tex`, `MagicFrens_Final_Audit.tex`, `MagicFrens_Independent_Audit_2026-09.tex`, three blind red-team write-ups and three exploit reports. Both of this pass’s findings turn out to be *descendants of prior fixes* — which is the most useful possible outcome of running blind, and the least flattering to the remediation process.

12.1.1 B-09 generalises C-02, which was patched one field wide

Prior finding **C-02** (`contracts/solidity/audit/CauldronAudit.tex:235`) reads:

“Un-validated `BrewSpec.nftSupply` bricks `relaunch()` — Critical — Permanent protocol freeze”

with `PoC_PoC_OversizedNftSupplyBricksRelaunch` and the note “`relaunch()` reverts forever; proposal not retired” (`CauldronAudit.tex:2095`). The audit even generalised the principle correctly at `CauldronAudit.tex:393`: “a proposal is an un-validated input to a constructor”.

The remedy bounded `nftSupply`. The five string fields of the same struct, reaching the same constructor, were left unbounded. B-09 is C-02 with the payload moved one field to the left. **The principle was correct, stated, and applied to exactly one of the six fields it covered.**

12.1.2 B-10 is a defect introduced by the Z-03 fix

Prior finding **Z-03** (`MagicFrens_Final_Audit.tex:664-698`) correctly identified that `_recomputeLeader` scanning every proposal ever submitted allowed a “permanent, permissionless halt of the eternal machine for a trivial cost”. The applied fix bounded the scan to 64. Its stated rationale is:

“The cached-leader fast path is untouched, so an established winner keeps winning however much spam follows it, and a fresh mandate is by definition among the most recent proposals.”

Both clauses are true, and together they are not sufficient. The first covers the *current* leader; the second covers a *fresh* mandate. Neither covers the *queued* mandate — one voted and settled before the spam arrived, which is what a guild that plans ahead produces — and `markConsumed` rebuilds the cache from the bounded scan at exactly the moment that mandate is needed. The fix converted an unbounded *gas* exposure into a bounded *correctness* one.

12.2 Which artifacts would have deflected this reviewer

Had the blinding failed, these specific passages would have redirected attention away from both findings:

Artifact	How it would have deflected
<code>MagicFrens_Final_Audit.tex:694-698</code>	States the scan bound is fixed and “post-fix the scan is flat”. Reading this, the reviewer would have marked the leader scan settled — and B-10 lives inside it.
<code>CauldronAudit.tex:235</code> (C-02 marked <i>Fixed</i>)	Marks proposal-input validation as a closed Critical, which is precisely the class B-09 reopens.
<code>test/attacks/B07_RelaunchTotality.t.sol:34-39</code>	Enumerates six sites that “clamp rather than revert” and asserts the remaining failures “were all in the steps that had NOT been given that treatment” — inviting the reader to treat the treated steps as done.
<code>cauldron/CauldronGovernor.sol:361-363</code> (survived stripping)	“an already-established winner keeps winning however much spam follows it; a fresh mandate simply has to be among the most recent proposals, <i>which it always is.</i> ” This comment is the false claim B-10 refutes, and it survived decontamination because stripping prose was out of scope.

12.3 Comments that were false when written

The commissioned discipline treats comments as unverified claims. Two failed:

1. `cauldron/CauldronGovernor.sol:361-363` — “a fresh mandate simply has to be among the most recent proposals, which it always is.” True of a fresh mandate; the queued mandate it omits is B-10.
2. `test/attacks/B07_RelaunchTotality.t.sol:19-20` — the property “`relaunch()` must never revert in a way that rolls back `markConsumed`” is asserted by a suite that stubs the governor, so the property is stated far more broadly than the tests establish it.

12.4 What the prior passes caught that this one did not

This pass examined a fraction of what the prior corpus covers. The prior work found and fixed defects in the perp mark price and staleness handling (Z-02), seeder rescue (Z-04), the L2 block

clock (Z-05), reserve ceiling breaches (Y-01), depth manipulation (Y-02), fee-asset routing (Q-01, B-01), non-ETH relaunch (B-05) and proposer-fee denomination (B-06). None of those areas was re-examined here, and nothing in this report should be read as re-validating them.

13 Threats to validity

1. **The commissioned scope is still not complete.** Of the seven risk areas named in the brief, four were examined: relaunch lifecycle blocking (B-09, B-10), attacker-influenced relaunch input (B-09, B-13), fee-reserve behaviour (B-13 and the reserve inventory), and oracle failure handling (B-11). *Still not examined adversarially:* per-generation holder claim guarantee; perp vault insolvency; supply conservation in mint paths; wrongful liquidation of a solvent position. The absence of findings in those areas is an absence of review, not evidence of safety.
2. **The graph exists but is partial.** Three of five clusters were extracted (321 functions). The registry, pool/seed and governance clusters were not, and the three derived views the brief asked for — value-flow, authority lattice, trust-boundary map — were never assembled as formal artifacts. Cross-contract edges were followed by hand where a finding required it, not enumerated systematically.
3. **Extraction was delegated to a smaller model.** The second dispatch ran on Sonnet rather than Opus to survive the rate limit. Every claim promoted from it into this report was independently re-verified by reading the cited lines — and two of the three claims promoted were *refuted* on that re-check (Section 8), which is a direct measure of why the verification step exists. Claims in the graph files that were not promoted here carry no such guarantee.
4. **Three perp-cluster leads were surfaced and not pursued.** The extraction recorded 11 unchecked blocks, a liquidation formula evaluated against a TWAP with a spot fallback, and a markSource that is an unconstrained CONFIGURABLE address. None was attacked. These sit directly on top of the two unexamined risk areas.
5. **Reviewer memory leaked prior findings.** Summaries of B-01, B-05 and B-06, with file and line references into the relaunch and fee paths, were in the reviewer’s persistent store and could not be unseen. B-09 was found in the relaunch path.
6. **One suite failure was environmental and is reported as such.** An intermediate run showed `invariant_PERP2_TotalEthMatchesDefinition` failing with an HTTP 429 from the public Sepolia RPC during fork setup. It passed on re-run. It is a rate-limit, not a defect — but it means invariant runs against the public endpoint are not reliably reproducible.
7. **Severity of B-09 depends on governance turnout.** It is rated High rather than Critical because the guild can out-vote a poisoned proposal. If an attacker holds a voting plurality it is Critical and unrecoverable. Turnout was not modelled.
8. **The block gas limit is a parameter, not a constant.** Measurements use 30M. The finding is not sensitive to that choice — replay costs more per byte than authoring — but the specific payload size required is.
9. **Off-chain scope was not touched.** The Ponder indexer, the Vercel serverless functions and the React frontend were not reviewed at all.
10. **Lead L-2 is DERIVED, not VERIFIED.** It is reasoned from the code and was not exercised.

14 Appendix — reproduction

14.1 Environment

```
cd contracts/solidity
export FORK_RPC=https://ethereum-sepolia-rpc.publicnode.com \
    POOL_MANAGER=0xE03A1074c86CFEdD5C142C4F04F1a1536e203543 \
    POSITION_MANAGER=0x429ba70129df741B2Ca2a85BC3A2a3328e5c09b4
FOUNDRY_PROFILE=cauldron forge build --sizes
FOUNDRY_PROFILE=cauldron forge test
```

14.2 The two proofs of concept

```
# All five proofs of concept
FOUNDRY_PROFILE=cauldron forge test --match-path "test/audit/B1[0-3]*.t.sol" -vv
FOUNDRY_PROFILE=cauldron forge test \
    --match-path "test/audit/B09_ProposalPayloadGasBrick.t.sol" -vv
```

B-09 and B-10 are in CauldronGovernor; B-11 in QuoteOracle; B-12 in QuoteRotator; B-13 in PoolOps.seedFunding. B-09's end-to-end case and B-11/B-12/B-13 differ in one respect worth noting for reproduction: B-11, B-12 and B-13 need *no fork*, because each targets a unit that can be exercised in isolation (seedFunding is an external library function, so it is directly callable). Only B-09's end-to-end rebirth needs the V4 PoolManager.

To observe each pre-fix failure, revert the corresponding hunk and re-run; the test_INVARIANT_* case in that file fails and the rest pass. This was done for B-09 and B-10 by neutralising the checks while keeping the declarations, so the tests still compiled — both invariants failed, confirming the regressions are not vacuous.

14.3 Decontamination statistics

Quantity	Value
Documents quarantined	63 (57 in audit/ trees, 6 elsewhere)
Source files edited by the stripper	86
Comment lines edited	486
Identifier tokens left in comments	0
Identifier tokens left in string literals	48 (unremovable)
Build size-table rows compared	1,138
Size-table MD5, real vs blind	identical (6e585e9c...)
Runtime bytecode delta, 3 largest contracts	53-byte metadata trailer only

14.4 Files changed

File	Change
cauldron/CauldronGovernor.sol	B-09 caps + B-10 runner-up (+449 bytes)
test/audit/B09_ProposalPayloadGasBrick.t.sol	new, 6 tests
test/audit/B10_ScanWindowErasure.t.sol	new, 5 tests